Restaurant Review Portal

Team DevDynasty

Version Control: Mechanism, Responsibility & Policy

1. Purpose

This document defines the version control strategy for the Restaurant Review Portal project. It specifies the tool selected, repository structure, branching model, commit conventions, responsibilities, and governing policy. The policy applies to all team members from project kick-off through to final submission.

2. Version Control Mechanism

2.1 Tool: Git with GitHub

The team will use Git as the distributed version control system, hosted on GitHub (private repository). GitHub is selected because:

- All team members have prior familiarity with Git and GitHub workflows.
- GitHub provides a free private repository suitable for academic use.
- Built-in Pull Request (PR) and code review tooling supports quality control.
- GitHub Issues can supplement the change management log.
- GitHub Actions can be used for automated testing (CI) if time permits.

2.2 Repository Structure

The repository will be organized as follows:

Directory / File	Contents
/docs	All project documentation (charter, plans, risk assessment, etc.)
/src/frontend	HTML, CSS, JavaScript for the customer-facing portal
/src/backend	Server-side code (PHP/Python/Node — to be confirmed at sprint 1)
/src/database	SQL schema scripts and seed data
/tests	Unit and integration test files
README.md	Project overview, setup instructions, and local run guide
.gitignore	Excludes IDE configs, node_modules, compiled assets, env files

2.3 Branching Model

The team will follow a simplified Git Flow model with three branch tiers:

Branch	Purpose	Who Creates
main	Stable, releasable code. Only updated via approved PRs from develop. Represents the current demo-ready state.	Repository Owner (once at setup)
develop	Integration branch. All feature branches merge here. Continuous integration tests run here.	Repository Owner (once at setup)
feature/<name>	One branch per task or feature (e.g. feature/restaurant-search, feature/review-submission). Short-lived; merged and deleted after review.	Individual developer

Hotfix branches (hotfix/<description>) may be created from main for critical bug fixes and merged back into both main and develop.

No team member may commit directly to main or develop and all changes must pass through a Pull Request.

3. Commit Conventions

All commits must follow the convention below to maintain a clean and readable history:

```
Format: <type>(<scope>): <short description>
```

Type	Usage
feat	A new feature or user-visible change (e.g. feat(search): add keyword filter for cuisine type)
fix	A bug fix (e.g. fix(moderation): resolve duplicate review submission)
docs	Documentation only changes (e.g. docs(readme): update setup instructions)
test	Adding or updating tests (e.g. test(backend): add unit test for price calculator)
refactor	Code restructuring with no functional change
chore	Build system, dependency updates, configuration

Commit messages must be written in the imperative mood ("Add feature" not "Added feature"). Each commit should represent one logical unit of work.

4. Responsibilities

Role	Responsibility
Repository Owner (designated member)	1. Creates and configures the GitHub repository. 2. Manages branch protection rules (require PR review before merge to main/develop). 3. Adds all team members as collaborators. 4. Resolves merge conflicts escalated by team members.

Role	Responsibility
All Developers	1. Create feature branches from develop for all work. 2. Write descriptive commit messages following the convention. 3. Open Pull Requests upon completion of a feature. 4. Review at least one peer PR per sprint; never force-push to shared branches.
Pull Request Reviewer (rotating per sprint)	1. Reviews PR code within 48 hours of submission. 2. Approves, requests changes, or flags for team discussion. 3. Responsible for merge after approval.
Scrum Master	1. Monitors that all team members are committing regularly. 2. Raises concerns about dormant branches or bypassed policies in weekly meetings. 3. Ensures docs/ is kept up to date after any document change.

5. Version Control Policy

5.1 Mandatory Rules

- All work must be committed to the repository and no work stored only locally.
- Feature branches must be branched from develop, never from main.
- A Pull Request requires at least one approving review before merging.
- No direct pushes to main or develop.
- Sensitive information (passwords, API keys, database credentials) must never be committed. Use environment variables and .env files listed in .gitignore.
- Commit at the end of every working session and partial work should be committed to the feature branch to avoid loss.

5.2 Pull Request Policy

- PR title must summarize the change and link to the relevant task in the project tracker.
- PR description must explain what was changed and why, and include testing steps.
- PRs must not introduce broken tests or failing builds.
- Stale PRs open more than one week without activity will be flagged by the Scrum Master.

5.3 Tagging and Releases

At the end of each sprint and at submission, the Repository Owner will create a Git tag on main using semantic versioning:

- v0.1.0 — End of Sprint 1 (initial working prototype)
- v0.2.0 — End of Sprint 2 (core features complete)
- v1.0.0 — Final submission

5.4 Conflict Resolution

If a merge conflict arises that a developer cannot resolve independently, they must:

1. Notify the team via the group messaging channel.
2. Not force-push or override the other contributor's work.
3. Arrange a short call with the other contributor to resolve together.
4. The Repository Owner has final authority if agreement cannot be reached.

5.5 Enforcement

Compliance with this policy is expected of all team members. Repeated non-compliance (e.g. direct pushes to protected branches, missing commit messages) will be noted in weekly meeting minutes and raised with the module teaching team if unresolved.